📖 `docs/REFERENCE.md` — WFI-AI-CFI TM Viewer Comprehensive Reference

A single-window guide covering APIDs, packet structure, software architecture, TM simulation, the ICD mapping, and how to extend everything.

Table of Contents

1. System Overview
2. Software Architecture
3. APID Registry
4. CCSDS Packet Construction
5. TM Dictionary — Subsystems, Cards, Parameters
6. RS-422 Link Telemetry
7. TM Ingestion Pipeline
8. Storage Architecture
9. The Simulator
10. UI Reference — Pages and Components
11. Hex Inspector & Packet Inspector Usage
12. REST & WebSocket API
13. Mapping TM → UI → Backend
14. The ICD & Interfaces
15. Extending the System
16. Placeholder — OEM Packet Format (Future)
17. Troubleshooting Map

---

1. System Overview

What this software is

The **WFI-AI-CFI TM Viewer** is the telemetry viewer and analyzer subsystem of the **WFI-AI-CFI SCC-SW** (Satellite Control Center Software). It is responsible for:

- **Receiving** TM from the WFI-AI-CFI instrument (via TCP, UDP, file, or DB)
- **Inspecting** raw and decoded packets
- **Storing** TM in a live stream (Redis) and a playback DB (Postgres)
- **Visualizing** live trends, alarms, link health
- **Replaying** historic windows for post-pass analysis

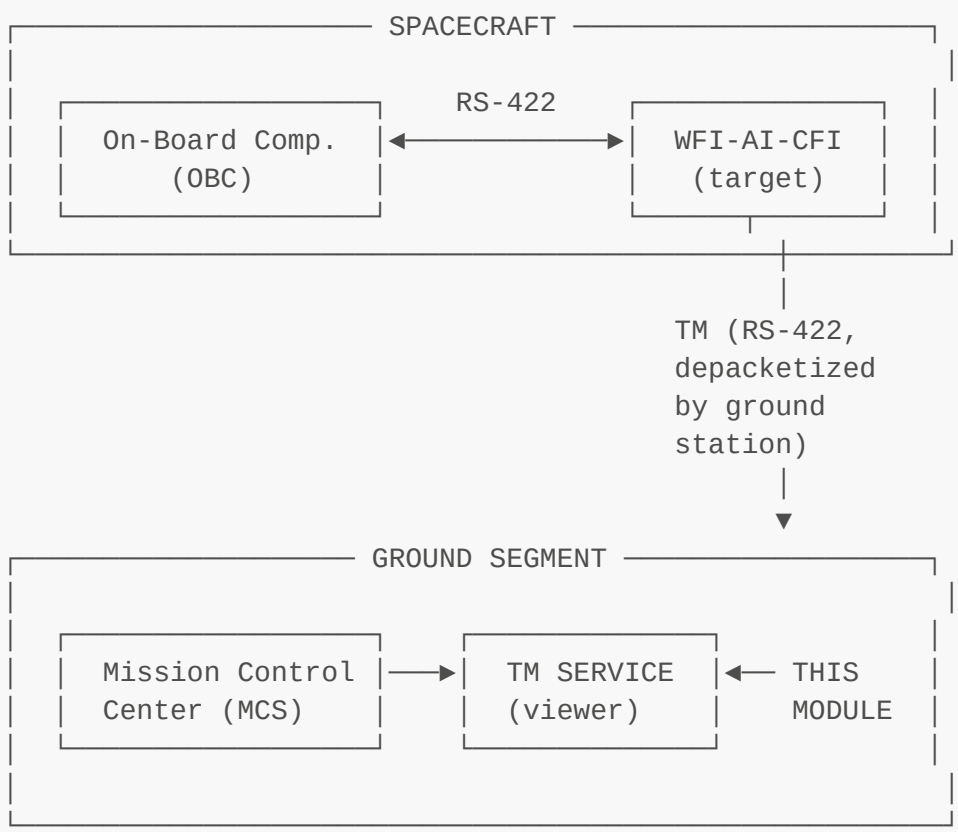
What it is *not*

- **Not** the Command Service (TC) — that's another engineer's scope
- **Not** the MPT (Mission Planning Tool) — also out of scope

- **Not** the Mission Control Station (MCS) or Mission Planning Service (MPS) — those belong to the *spacecraft* segment

The physical target

The **WFI-AI-CFI** is a **Customer Furnished Instrument** — an autonomous imaging and AI-processing unit that is *separate* from the host spacecraft. It has its own internal subsystems, its own power conditioning, and its own instrument controller.



TM arrives at the TM viewer from the MCS (Mission Control Center) over a TCP or UDP socket, gets depacketized, decoded against the TM dictionary, and displayed.

The instrument's internal subsystems

Subsystem	Purpose	Redundancy
Camera Subsystem	Focal plane, CCD readout, imaging electronics	Single (no redundancy)
CDPM (Camera Data Processing Module)	FPGA processing, AI inference, thermal control	Primary + Redundant
CTPU (Camera Thermal & Power Unit)	Power conditioning, DC-DC conversion, thermal control	Primary + Redundant
Unit HK	Top-level instrument housekeeping (mode, uptime, active chain)	—

Each subsystem contains one or more **cards** (PCBs). Every card emits its own TM packet. All inter-subsystem links are **RS-422** and are themselves TM sources.

---

2. Software Architecture

Repository layout

```
wfi-ai-cfi-tm-viewer/
├── backend/                        # FastAPI + Python 3.11
│   ├── app/
│   │   ├── main.py                 # FastAPI entry + lifespan
│   │   ├── config.py               # Pydantic Settings
│   │   ├── api/                    # HTTP + WebSocket routers
│   │   ├── tm_service/             # YOUR SUBSYSTEM
│   │   │   ├── ingestion/          # TCP, UDP, file, DB sources
│   │   │   ├── inspector/          # CCSDS parse, decode, CRC
│   │   │   ├── storage/            # Redis + Postgres writers
│   │   │   ├── playback/           # DB replay sessions
│   │   │   └── registry.py         # YAML → in-memory map
│   │   ├── tm_dictionary/          # parameters.yaml, links.yaml
│   │   ├── db/models/              # SQLAlchemy ORM
│   │   ├── schemas/                # Pydantic DTOs
│   │   └── sim/                    # Simulator (fills gap until ICD final)
│   ├── scripts/                    # Dev utilities
│   └── tests/
├── frontend/                       # React 18 + Vite + TS
│   └── src/
│       ├── components/             # Presentational
│       ├── pages/                  # One per route
│       ├── hooks/                  # Data hooks
│       ├── store/                  # Zustand stores
│       ├── lib/                    # Utils, api, colors, format
│       └── types/                  # Shared DTOs (mirror backend)
└── docker-compose.yml              # Postgres + Redis
```

Backend modules

Module	Responsibility
`app/main.py`	FastAPI app, lifespan, ingestion bootstrap
`app/config.py`	Reads `.env`, exposes `settings` singleton
`app/api/routes_*.py`	HTTP endpoints (`/api/meta`, `/api/history`, …)
`app/api/ws_telemetry.py`	WS `/ws/telemetry` (live) and `/ws/playback` (replay)

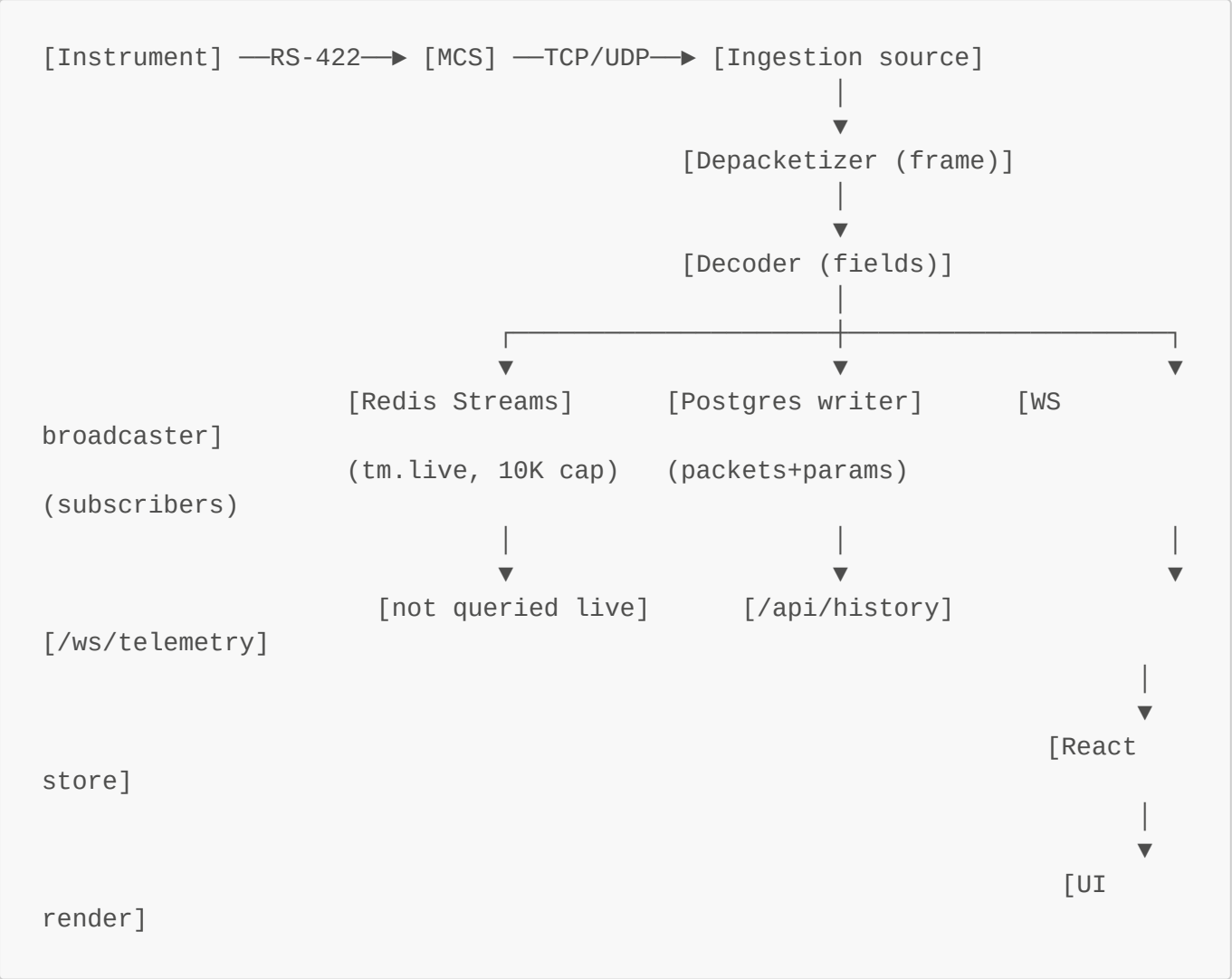
Module	Responsibility
`app/tm_service/registry.py`	Loads YAML, validates offsets, exposes `apid_map`
`app/tm_service/ingestion/manager.py`	Owns sources, fans out to subscribers, holds writer
`app/tm_service/ingestion/tcp_source.py`	TCP server on :9100
`app/tm_service/ingestion/udp_source.py`	UDP listener on :9200
`app/tm_service/ingestion/depacketizer.py`	Stream → frames using length field
`app/tm_service/inspector/ccsds.py`	Parse 7-byte primary header
`app/tm_service/inspector/decoder.py`	Field extraction per APID
`app/tm_service/inspector/crc.py`	CRC-16-CCITT
`app/tm_service/storage/writer.py`	Dual-write: Redis Streams + Postgres
`app/tm_service/playback/player.py`	Time-based replay iterator
`app/sim/spacecraft_model.py`	Sim state + tick
`app/sim/packet_builder.py`	State → CCSDS frames
`app/sim/tcp_emitter.py`	Feeds TCP source locally

Frontend modules

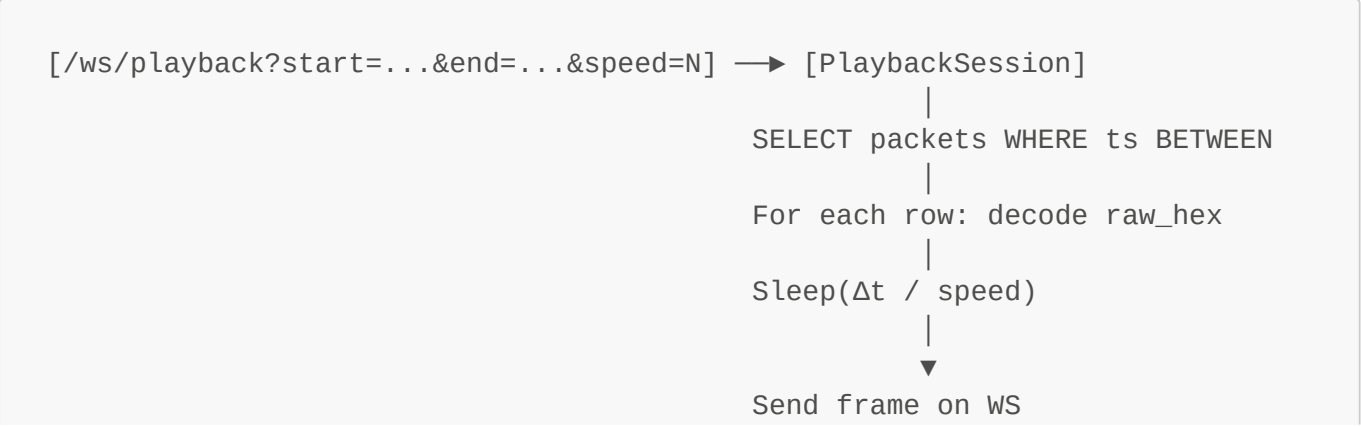
Directory	Responsibility
`src/components/layout/`	`StatusBar`, `Sidebar`
`src/components/live/`	`KpiCards`, `LiveChart`, `PacketDecoder`, `AlarmRail`, `LinkHealthStrip`
`src/components/inspector/`	`HexViewer`, `DecodedFieldsList`, `FilterBar`, `UploadDumpButton`
`src/components/trends/`	`MultiSeriesChart`, `TrendSparkTable`, `ZoomControls`
`src/components/storage/`	`ExportButton`, `WindowList`
`src/components/playback/`	`TimelineScrubber`, `SpeedControl`
`src/pages/`	One component per route: `LivePage`, `TrendsPage`, `StoragePage`, `AlarmsPage`, `InspectorPage`, `PlaybackPage`, `LinksPage`
`src/store/telemetryStore.ts`	Live packet buffer, latest-by-APID, alarms, link state
`src/store/metaStore.ts`	TM dictionary cache from `/api/meta`
`src/store/filterStore.ts`	Sidebar filters (subsystem, card, HW/SW)

Directory	Responsibility
src/store/uiStore.ts	Active page + Live/Playback mode
src/hooks/useTelemetryStream.ts	WS client with reconnect
src/hooks/useMeta.ts	Fetch /api/meta once
src/hooks/useAlarms.ts	Derived alarm stats

Data flow (Live)



Data flow (Playback)



```
                                                  |
                                                  ▼
                                          [React store ingest]
```

---

3. APID Registry

Every card and every RS-422 link has a **unique APID** (Application Process Identifier), 11 bits wide. The APID tells the decoder which dictionary entry to use.

Allocated APIDs

APID (hex)	Dec	Subsystem	Card / Link	HW Class
0x100	256	CAMERA	CAM_ELEC — Camera Electronics Card	HW
0x110	272	CDPM_P	CDPM_P_FPGA — FPGA Processing Card	HW
0x111	273	CDPM_P	CDPM_P_AI — AI Board	HW
0x112	274	CDPM_P	CDPM_P_THCC — Thermal Control Card	HW
0x120	288	CDPM_R	CDPM_R_FPGA — FPGA Processing Card	HW
0x121	289	CDPM_R	CDPM_R_AI — AI Board	HW
0x130	304	CTPU_P	CTPU_P_PWR — Power Card	HW
0x131	305	CTPU_P	CTPU_P_PPC — Power Processing Card	HW
0x140	320	CTPU_R	CTPU_R_PWR — Power Card	HW
0x141	321	CTPU_R	CTPU_R_PPC — Power Processing Card	HW
0x150	336	UNIT	UNIT_HK — Unit Housekeeping	SW
0x200	512	LINKS	CAM_CDPM_P — Camera ↔ CDPM (P)	HW
0x201	513	LINKS	CAM_CDPM_R — Camera ↔ CDPM (R)	HW
0x202	514	LINKS	CDPM_P_CTPU_P — CDPM (P) ↔ CTPU (P)	HW
0x203	515	LINKS	CDPM_R_CTPU_R — CDPM (R) ↔ CTPU (R)	HW
0x204	516	LINKS	CTPU_P_SC — CTPU (P) ↔ Spacecraft	HW
0x205	517	LINKS	CTPU_R_SC — CTPU (R) ↔ Spacecraft	HW
0x206	518	LINKS	CDPM_CROSSSTRAP — CDPM (P) ↔ CDPM (R)	HW

APID allocation rules

- **0x100 – 0x1FF** — subsystem cards and unit HK
- **0x200 – 0x2FF** — RS-422 link telemetry
- **0x300 – 0x3FF** — reserved for future: TC echo, memory dumps, image metadata
- **0x400+** — reserved for future expansion

APIDs are assigned in `backend/app/tm_dictionary/parameters.yaml` (for cards) and `links.yaml` (for links). The registry validates on load: no duplicate APIDs allowed.

---

4. CCSDS Packet Construction

Frame layout (implemented)

Every TM packet has the following structure on the wire:

```
┌──────────────────────────────────────────────────────────┐
│                  CCSDS PRIMARY HEADER (7 bytes)          │
├────────┬──────────────────┬──────────────────┬───────────┤
│ Byte   │ Field            │ Bits             │ Meaning   │
├────────┼──────────────────┼──────────────────┼───────────┤
│ 0      │ CC               │ 8                │ 0x08      │
│ 1-2    │ APID + flags     │ 11 APID + 5 flag │ big-endian│
│ 3-4    │ SEQ + flags      │ 14 SEQ + 2 flag  │ big-endian│
│ 5-6    │ Length - 1       │ 16               │ payload-1 │
├────────┴──────────────────┴──────────────────┴───────────┤
│                     PAYLOAD (N bytes)                    │
│  (APID-specific, layout defined in parameters.yaml)      │
├──────────────────────────────────────────────────────────┤
│                  CRC-16-CCITT (2 bytes)                  │
│  (calculated over header + payload, big-endian)          │
└──────────────────────────────────────────────────────────┘
```

Field details

CC (Command/Control byte) — always `0x08` in our simplified spec. Real CCSDS uses this for version/type/sec-header flags; we keep it simple.

APID — 11-bit identifier. Stored in the lower 11 bits of bytes 1–2. Byte 1's top 5 bits are flags (unused, set to `0xC0` for "secondary header absent, packet type = telemetry").

Sequence counter — 14-bit rolling counter, wraps at 16384. Top 2 bits flag (unused). Used to detect packet loss.

Length — payload length minus 1. Standard CCSDS convention. Payload of 24 bytes → `length = 23`.

Payload — the actual TM data. Layout per APID is in the dictionary.

CRC — CRC-16-CCITT (poly `0x1021`, init `0xFFFF`) over header + payload. Big-endian.

Python encode example

```python
import struct
from app.tm_service.inspector.crc import crc16_ccitt

def build_frame(apid: int, payload: bytes, seq: int) -> bytes:
```

```
    length = len(payload) - 1
    header = struct.pack(">BHHH", 0x08, apid | 0xC000, seq | 0xC000,
length)
    crc = crc16_ccitt(header + payload)
    return header + payload + struct.pack(">H", crc)
```

Python decode example

```
from app.tm_service.inspector.ccsds import parse_ccsds_header
from app.tm_service.inspector.crc import crc16_ccitt

def decode_frame(raw: bytes):
    hdr = parse_ccsds_header(raw)
    payload_end = 7 + hdr.length + 1
    payload = raw[7:payload_end]
    crc_rx = struct.unpack(">H", raw[payload_end:payload_end+2])[0]
    crc_ok = crc16_ccitt(raw[:payload_end]) == crc_rx
    return hdr, payload, crc_ok
```

Field packing convention

Numeric fields are **big-endian**. Float fields are IEEE-754 single precision (`f32`) or double (`f64`).

YAML type	struct fmt	Size	Range
`u8`	`B`	1	0 – 255
`u16`	`H`	2	0 – 65535
`u32`	`I`	4	0 – 4.29e9
`i8`	`b`	1	−128 – 127
`i16`	`h`	2	−32768 – 32767
`i32`	`i`	4	−2.15e9 – 2.15e9
`f32`	`f`	4	IEEE-754 single
`f64`	`d`	8	IEEE-754 double

Fields may be packed in **any order** as long as offsets don't overlap. Gaps between fields are allowed (padded with zeros). The registry's `_validate()` rejects overlapping fields.

4.5 Packet Structure Sizes — Complete Byte-Level Reference

Two formats, two header layouts

The TM Viewer supports **two** on-wire packet formats, selected by the `TM_DECODER` environment variable:

Format	TM_DECODER	Primary header	Length field semantics	Total packet size
Legacy custom	custom	7 bytes	payload_len − 1	10 + payload_len
Standard CCSDS / PUS-C	pus	6 bytes	total_len − 7 (CCSDS)	7 + length_field

Both formats append a **2-byte CRC-16-CCITT** (poly 0x1021, init 0xFFFF, big-endian) after the payload.

Common structure

```
┌──────────────────────────────────────────────────────┐
│  PRIMARY HEADER    (6 or 7 bytes, format-dependent)  │
├──────────────────────────────────────────────────────┤
│  SECONDARY HEADER  (0 or 13/14 bytes, format-dependent)│
├──────────────────────────────────────────────────────┤
│  PAYLOAD / APP DATA (N bytes, APID-specific)         │
├──────────────────────────────────────────────────────┤
│  CRC-16-CCITT      (2 bytes, big-endian)             │
└──────────────────────────────────────────────────────┘
```

Format A — Legacy custom (7-byte header, TM_DECODER=custom)

Used by the internal simulator and the earlier ICD draft.

Offset	Size	Field	Description
0	1	CC	Command/Control byte — always 0x08
1–2	2	APID + flags	Bits 0–10 = APID, bits 11–15 = flags (0xC0)
3–4	2	SEQ + flags	Bits 0–13 = sequence count, bits 14–15 = flags (0xC0)
5–6	2	Length	payload_len − 1 (big-endian)
7	N	Payload	APID-specific — layout from parameters.yaml
7+N	2	CRC-16-CCITT	Big-endian, over header + payload

Total: 7 + N + 2 = 9 + N bytes.

Depacketizer formula:

```
length_field = struct.unpack_from(">H", buf, 5)[0]
payload_len  = length_field + 1
total        = 7 + payload_len + 2
```

Format B — Standard CCSDS + PUS-C (6-byte header, TM_DECODER=pus)

Used by the `send_pus_tm_apids.sh` script and by real PUS-compliant sources.

B.1 CCSDS Primary Header (6 bytes)

Offset	Size	Field	Bits	Description
0	1	Version / Type / SecHdrFlag / APID_hi	3 + 1 + 1 + 3	Version=0, Type=0 (TM), SecHdrFlag=1, top 3 bits of APID
1	1	APID_lo	8	Low 8 bits of the 11-bit APID
2	1	SeqFlags / SeqCount_hi	2 + 6	SeqFlags=0b11 (unsegmented), top 6 bits of count
3	1	SeqCount_lo	8	Low 8 bits of 14-bit sequence count
4–5	2	Packet Data Length	16	`total_packet_len - 7` (big-endian)

B.2 PUS-C Secondary Header (14 bytes)

Used with a **7-byte CUC timestamp** (the standard short format).

Offset (in sec-hdr)	Size	Field	Notes
0	1	PUS Version + AckFlags	Version=2 (`0x20`), Ack=0
1	1	Service Type	0–255
2	1	Service Subtype	0–255
3–4	2	Message Type Counter	Big-endian, wraps at 65535
5–6	2	Destination ID	Big-endian, 0 = broadcast
7–10	4	Coarse Time	Unix seconds (UInt32)
11–13	3	Fine Time	$1/2^{24}$ second units (UInt24)

Total secondary header: 14 bytes (1+1+1+2+2+4+3).

B.3 Full PUS-C TM Packet Layout

Offset	Size	Field
0–5	6	CCSDS primary header
6–19	14	PUS-C secondary header (7-byte CUC timestamp)
20	N	Application data (payload)
20+N	2	CRC-16-CCITT (big-endian)

Total: `6 + 14 + N + 2 = 22 + N` bytes.

Depacketizer formula:

```
length_field = struct.unpack_from(">H", buf, 4)[0]
total        = length_field + 7
```

Depacketizer dispatch

`backend/app/tm_service/ingestion/format_map.py` maps `TM_DECODER` to the correct header length:

```
HEADER_LEN_BY_DECODER = {
    "custom": 7,
    "pus":    6,
    "oem":    6,   # placeholder
}
```

`StreamDepacketizer(header_len=…)` then branches on that to apply the correct length formula.

Per-APID packet sizes (custom format)

Under `TM_DECODER=custom`, each APID has a fixed payload layout in `parameters.yaml`. Here are the exact on-wire sizes:

APID	Subsystem / Card	Payload (bytes)	Primary hdr	CRC	Total frame
`0x100` CAM_ELEC	Camera Electronics	38	7	2	**47 B**
`0x110` CDPM_P_FPGA	FPGA Processing	36	7	2	**45 B**
`0x111` CDPM_P_AI	AI Board	31	7	2	**40 B**
`0x112` CDPM_P_THCC	Thermal Control	22	7	2	**31 B**
`0x120` CDPM_R_FPGA	FPGA Processing	36	7	2	**45 B**
`0x121` CDPM_R_AI	AI Board	31	7	2	**40 B**
`0x130` CTPU_P_PWR	Power Card	31	7	2	**40 B**
`0x131` CTPU_P_PPC	Power Processing	30	7	2	**39 B**
`0x140` CTPU_R_PWR	Power Card	31	7	2	**40 B**
`0x141` CTPU_R_PPC	Power Processing	30	7	2	**39 B**
`0x150` UNIT_HK	Unit Housekeeping	17	7	2	**26 B**
`0x200`–`0x206` LINKS (7)	RS-422 link health	21	7	2	**30 B each**

> **Payload size is derived from the highest `offset + type_size` in the card's field list.** The numbers above are computed from the current `parameters.yaml`. If you add/remove fields, recompute by taking `max(offset + type_size)`.

Per-APID packet sizes (PUS-C format)

Under `TM_DECODER=pus`, the primary header shrinks to 6 bytes but a **14-byte secondary header** is added:

APID	Application data (bytes)	Primary hdr	Secondary hdr	CRC	Total frame
`0x100`	38	6	14	2	**60 B**
`0x110`	36	6	14	2	**58 B**
`0x111`	31	6	14	2	**53 B**
`0x112`	22	6	14	2	**44 B**
`0x120`	36	6	14	2	**58 B**
`0x121`	31	6	14	2	**53 B**
`0x130`	31	6	14	2	**53 B**
`0x131`	30	6	14	2	**52 B**
`0x140`	31	6	14	2	**53 B**
`0x141`	30	6	14	2	**52 B**
`0x150`	17	6	14	2	**39 B**
`0x200`–`0x206` (7)	21	6	14	2	**43 B each**

> **Note:** The 6-byte CCSDS primary header replaces the 7-byte custom header (`−1 byte`), but the 14-byte PUS secondary header is added (`+14 bytes`). Net difference vs custom: **+13 bytes per packet**.

What the `send_pus_tm_apids.sh` script emits

The bash script you're using generates **PUS-C Format B** packets with the following characteristics:

```
CCSDS primary header ........... 6 bytes
PUS-C secondary header ......... 14 bytes  (7-byte CUC timestamp)
Application data ............... 4 bytes   ("DEADBEEF" by default)
CRC-16-CCITT ................... 2 bytes
──────────────────────────────────────────
TOTAL per packet ............... 26 bytes
```

CCSDS length field value: `26 − 7 = 19` → `0x0013`, matching the wire capture.

Worked example — decoding a real UDP packet

Hex bytes from Wireshark (26 bytes):

```
0a 05 c0 91 00 13 20 03 19 00 91 00 00 6a cc 8c 07 3b 5b 9c de ad be ef a4
56
```

Offset	Bytes	Field	Value
0–1	`0a 05`	Version/Type/SecHdr/APID_hi + APID_lo	Version=0, Type=0, SecHdr=1, APID=`0x205`
2–3	`c0 91`	SeqFlags + SeqCount	Flags=0b11, Count=145
4–5	`00 13`	Packet Data Length	`0x0013 = 19` → total = 19 + 7 = 26 ✅
6	`20`	PUS Version + Ack	Version=2, Ack=0
7	`03`	Service Type	3 (housekeeping)
8	`19`	Service Subtype	25
9–10	`00 91`	Message Counter	145
11–12	`00 00`	Destination ID	0
13–16	`6a cc 8c 07`	Coarse Time	`0x6ACC8C07` = 1,790,036,487 (Unix seconds)
17–19	`3b 5b 9c`	Fine Time	UInt24 fractional seconds
20–23	`de ad be ef`	Application Data	`DEADBEEF`
24–25	`a4 56`	CRC-16-CCITT	Big-endian

`spacepackets` note: `PusTelemetry.unpack(data=..., timestamp_len=7)` parses the CCSDS + PUS headers correctly, exposes the application data as `tm.tm_data` (or the alias `tm.source_data`), and returns a CRC-validated object.

Quick size cheat sheet

Concern	Value
Legacy custom frame	`9 + payload_len` bytes
CCSDS + PUS-C frame	`22 + app_data_len` bytes
Overhead (custom)	9 bytes fixed
Overhead (PUS-C)	22 bytes fixed
Header-to-CRC ratio (typical)	~40–50 % overhead at 21–38 B payloads
Max payload (both)	65535 bytes (16-bit length field)
Padding	None — all fields byte-aligned, no packing flags

Reference — where these numbers come from

Source	File
Legacy header layout	`backend/app/tm_service/ingestion/depacketizer.py` — `header_len=7` branch
CCSDS header layout	`backend/app/tm_service/inspector/ccsds.py` and CCSDS 133.0-B-1
PUS-C secondary header	ECSS-E-ST-70-41C, and `spacepackets.ecss.tm.PusTm`
Field sizes	`backend/app/tm_service/registry.py` `STRUCT_SIZE` map
Payload sizes	Derived from `backend/app/tm_dictionary/parameters.yaml`
Length formulas	`StreamDepacketizer.feed()` — branches on `header_len`

5. TM Dictionary — Subsystems, Cards, Parameters

File: `backend/app/tm_dictionary/parameters.yaml`

This is the single source of truth. Edit it and restart the backend to change the TM layout. The registry reloads on process start.

Structure

```
subsystems:
  <SUBSYSTEM_KEY>:
    label: "Human Readable Name"
    color: "#RRGGBB"        # used for UI accents
    cards:
      <CARD_KEY>:
        label: "Card Name"
        apid: 0xXYZ
        hw_class: HW         # HW or SW
        fields:
          - { name: <id>, offset: <bytes>, type: <t>, unit: "<u>", ltl: .., lth: .., htl: .., hth: .. }
```

The 6 subsystems

Key	Label	Cards	Color
`CAMERA`	Camera Subsystem	`CAM_ELEC`	Cyan `#00E5FF`
`CDPM_P`	CDPM (Primary)	`CDPM_P_FPGA`, `CDPM_P_AI`, `CDPM_P_THCC`	Green `#22C55E`
`CDPM_R`	CDPM (Redundant)	`CDPM_R_FPGA`, `CDPM_R_AI`	Dark Green `#16A34A`
`CTPU_P`	CTPU (Primary)	`CTPU_P_PWR`, `CTPU_P_PPC`	Amber `#FFB020`
`CTPU_R`	CTPU (Redundant)	`CTPU_R_PWR`, `CTPU_R_PPC`	Dark Amber `#F59E0B`

Key	Label	Cards	Color
UNIT	Unit / Mode	UNIT_HK	Grey #8B95A9

Full parameter list per card

CAM_ELEC — Camera Electronics Card (0x100)

Field	Off	Type	Unit	LTL	LTH	HTL	HTH	Meaning
cam_fpa_temp	0	f32	°C	−40	−5	15	20	Focal plane array temperature
cam_ccd_temp	4	f32	°C	−30	−10	25	30	CCD detector temperature
cam_ccd_bias_v	8	f32	V	4.5	—	5.5	—	CCD bias voltage
cam_clock_locked	12	u8	—	—	—	—	—	PLL lock (0=UNLOCKED, 1=LOCKED)
cam_exposure_time	13	f32	ms	0.1	—	5000	—	Configured integration time
cam_frame_counter	17	u32	cnt	—	—	—	—	Frames read out since boot
cam_data_rate	21	f32	Mbps	0	—	800	—	Output link data rate
cam_cec_current	25	f32	A	0	—	3.5	—	Board input current
cam_cec_voltage	29	f32	V	27.5	—	28.5	—	Board input voltage
cam_cec_temp	33	f32	°C	−20	—	60	—	Board PCB temperature
cam_lvds_locked	37	u8	—	—	—	—	—	LVDS link (0=DOWN, 1=UP)

CDPM_P_FPGA — FPGA Processing Card (0x110)

Field	Off	Type	Unit	Limits
fpga_die_temp	0	f32	°C	−20 / 0 / 75 / 85
fpga_core_voltage	4	f32	V	0.95 – 1.05
fpga_aux_voltage	8	f32	V	1.75 – 1.85
fpga_io_voltage	12	f32	V	3.2 – 3.4
fpga_current	16	f32	A	0 – 5
fpga_ddr_used_mb	20	u32	MB	0 – 4096
fpga_frame_fifo_ovf	24	u32	cnt	—

Field	Off	Type	Unit	Limits
fpga_processing_latency_ms	28	f32	ms	0 – 500
fpga_watchdog_reset_count	32	u32	cnt	—

CDPM_P_AI — AI Board (0x111)

Field	Off	Type	Unit	Limits
ai_soc_temp	0	f32	°C	−20 / 0 / 85 / 95
ai_soc_power_w	4	f32	W	0 – 40
ai_core_util_pct	8	f32	%	0 – 100
ai_mem_util_pct	12	f32	%	0 – 100
ai_inference_latency_ms	16	f32	ms	0 – 200
ai_frames_processed	20	u32	cnt	—
ai_detections_count	24	u32	cnt	—
ai_model_id	28	u16	—	—
ai_health_status	30	u8	—	0=OK, 1=WARN, 2=FAULT

CDPM_P_THCC — Thermal Control Card (0x112)

Field	Off	Type	Unit	Limits
thcc_heater_pwm_pct	0	f32	%	0 – 100
thcc_heater_current	4	f32	A	0 – 4
thcc_cam_plate_temp	8	f32	°C	−30 – 40
thcc_cdpm_plate_temp	12	f32	°C	−20 – 60
thcc_setpoint	16	f32	°C	−20 – 30
thcc_control_loop_state	20	u8	—	0=OFF, 1=ON, 2=FAULT
thcc_thermistor_ok	21	u8	—	bitfield

CDPM_R_FPGA (0x120) — same layout as CDPM_P_FPGA

CDPM_R_AI (0x121) — same layout as CDPM_P_AI

CTPU_P_PWR — Power Card (0x130)

Field	Off	Type	Unit	Limits
pwr_bus_42v_in_v	0	f32	V	40 – 44

Field	Off	Type	Unit	Limits
pwr_bus_42v_in_i	4	f32	A	0 – 15
pwr_out_42v_cam_v	8	f32	V	40 – 44
pwr_out_42v_cam_i	12	f32	A	0 – 10
pwr_out_28v_bus_v	16	f32	V	26 – 30
pwr_out_28v_bus_i	20	f32	A	0 – 8
pwr_uv_ov_status	24	u8	—	bitfield
pwr_overcurrent_fault	25	u8	—	0/1
pwr_efuse_state	26	u8	—	0=OFF, 1=ON, 2=TRIP
pwr_temp_pcb	27	f32	°C	−20 – 85

CTPU_P_PPC — Power Processing Card (0x131)

Field	Off	Type	Unit	Limits
ppc_primary_bus_v	0	f32	V	27.5 – 28.5
ppc_primary_bus_i	4	f32	A	0 – 6
ppc_secondary_bus_v	8	f32	V	4.8 – 5.2
ppc_dcdc_temp	12	f32	°C	−20 – 100
ppc_efficiency_pct	16	f32	%	70 – 95
ppc_switch_freq_khz	20	f32	kHz	480 – 520
ppc_fault_flags	24	u16	—	bitfield
ppc_uptime_s	26	u32	s	—

CTPU_R_PWR (0x140) — same layout as CTPU_P_PWR

CTPU_R_PPC (0x141) — same layout as CTPU_P_PPC

UNIT_HK — Unit Housekeeping (0x150)

Field	Off	Type	Unit	Enum
unit_mode	0	u8	—	0=SAFE, 1=IDLE, 2=IMAGING, 3=CALIB, 4=FAULT
unit_uptime_s	1	u32	s	—
unit_active_cdpm	5	u8	—	0=PRIMARY, 1=REDUNDANT
unit_active_ctpu	6	u8	—	0=PRIMARY, 1=REDUNDANT
unit_obc_time_corr	7	f64	s	—

Field	Off	Type	Unit	Enum
unit_last_tc_seq	15	u16	—	—

6. RS-422 Link Telemetry

Every RS-422 link is a first-class TM source. Each link has an APID and a fixed 6-field layout.

File: backend/app/tm_dictionary/links.yaml

```
links:
  - id: CAM_CDPM_P
    label: "Camera <-> CDPM (P)"
    apid: 0x200
    fields:
      - { name: link_state,      offset: 0,  type: u8,  unit: "-", enum:
{0: "DOWN", 1: "UP", 2: "DEGRADED"} }
      - { name: crc_err_count,   offset: 1,  type: u32, unit: "cnt" }
      - { name: frame_err_count, offset: 5,  type: u32, unit: "cnt" }
      - { name: rx_rate_kbps,    offset: 9,  type: f32, unit: "kbps" }
      - { name: tx_rate_kbps,    offset: 13, type: f32, unit: "kbps" }
      - { name: latency_us,      offset: 17, type: f32, unit: "us" }
  # ... 6 more links, same schema
```

The 7 links

APID	ID	Between	Purpose
0x200	CAM_CDPM_P	Camera ↔ CDPM (P)	Primary image data path
0x201	CAM_CDPM_R	Camera ↔ CDPM (R)	Redundant image path
0x202	CDPM_P_CTPU_P	CDPM (P) ↔ CTPU (P)	Primary data + control
0x203	CDPM_R_CTPU_R	CDPM (R) ↔ CTPU (R)	Redundant data + control
0x204	CTPU_P_SC	CTPU (P) ↔ Spacecraft	Primary TM/TC with host S/C
0x205	CTPU_R_SC	CTPU (R) ↔ Spacecraft	Redundant TM/TC with host
0x206	CDPM_CROSSSTRAP	CDPM (P) ↔ CDPM (R)	Cross-strap health check

Link field meanings

Field	Meaning
link_state	0=DOWN, 1=UP, 2=DEGRADED
crc_err_count	Cumulative CRC errors since power-on
frame_err_count	Cumulative framing errors

Field	Meaning
`rx_rate_kbps`	Measured receive rate
`tx_rate_kbps`	Measured transmit rate
`latency_us`	Round-trip latency in microseconds

Design note: All 7 links currently report the same values because the simulator has a single `link_state` attribute. In production each link's TM comes from its own UART controller with its own counters.

---

7. TM Ingestion Pipeline

Supported sources

Source	Port	Mode	Use
TCP	9100	Server (we listen)	MCS pushes frames to us
UDP	9200	Server (we listen)	Fire-and-forget test or MCS bridge
File	—	`scripts/load_dump.py`	Post-pass replay from a `.bin`/`.dump`
DB	—	Playback sessions	Historical replay

Depacketizer

Given a byte stream, the depacketizer slices out complete frames. It supports **two header layouts**, selected by the `header_len` parameter:

Custom format (`header_len=7`, `TM_DECODER=custom`)

1. Accumulate incoming bytes in a buffer
2. Read `length` from bytes 5–6 (payload length – 1)
3. Total frame = `7 + (length + 1) + 2` = **`length + 10`** bytes
4. If buffer has a full frame → emit it, delete from buffer
5. If `length` is bogus (> 65535 or would require overlap) → shift by 1 byte (resync)

CCSDS / PUS-C format (`header_len=6`, `TM_DECODER=pus`)

1. Accumulate incoming bytes in a buffer
2. Read `length` from bytes 4–5 (CCSDS "Packet Data Length")
3. Total frame = `length + 7` bytes (CCSDS convention: `length = total_len − 7`)
4. If buffer has a full frame → emit it, delete from buffer
5. If `length` is bogus → shift by 1 byte (resync)

Length formula summary

Format	`header_len`	Length field semantics	Total packet size
Legacy custom	7	`payload_len − 1`	`length_field + 10`

Format	header_len	Length field semantics	Total packet size
Standard CCSDS	6	total_len − 7	length_field + 7

The header length is chosen at startup via `format_map.header_len_for(settings.tm_decoder)`. The `StreamDepacketizer` is used by **both** TCP source and every UDP datagram (a fresh instance per datagram to guarantee message-boundary alignment).

Decode step

Every emitted frame runs through `decoder.decode(raw, ts)`:

1. Parse the 7-byte header → get APID, seq, length
2. Look up APID in registry → get card spec + field list
3. Slice payload at `raw[7 : 7+length+1]`
4. For each field: unpack at `offset`, apply enum normalization, compute status vs limits
5. Return a `DecodedPacket` dict

Status classification

For each field with limits, the decoder returns one of:

Status	Condition
CRIT	value < LTL or value > HTL
WARN	value < LTH or value > HTH
OK	within normal operating range

Fields without limits always return `OK`.

Fan-out

Once decoded, the packet is written to Redis + Postgres and broadcast to every WebSocket subscriber. Failures at any stage are logged and swallowed — one bad packet never kills the pipeline.

---

8. Storage Architecture

Two databases, two purposes.

Redis (Live stream DB)

- **What:** Redis Stream `tm.live`
- **Format:** JSON blobs of `DecodedPacket`
- **Cap:** 10,000 entries (`maxlen=10000`, `approximate=True`)
- **Purpose:** sub-100ms read for the WS broadcaster or a caching layer
- **Retention:** last ~10K packets (~5 minutes at 30 pkt/s)

Inspect:

```
docker exec -it tm-redis redis-cli XLEN tm.live
docker exec -it tm-redis redis-cli XREVRANGE tm.live + - COUNT 5
```

Postgres (Playback DB)

- **What:** durable tables `packets` + `parameters`
- **Purpose:** query by time window, export CSV, replay
- **Retention:** unbounded (you manage pruning)

Schema:

`packets` — one row per TM packet

Column	Type	Index
`id`	bigint PK	—
`ts`	timestamptz	yes
`apid`	int	yes
`subsystem`	varchar(32)	yes
`card`	varchar(32)	yes
`seq`	int	—
`crc_ok`	bool	—
`raw_hex`	text	—
`payload_hex`	text	—

`parameters` — one row per (packet, field)

Column	Type	Index
`id`	bigint PK	—
`packet_id`	bigint FK	yes
`name`	varchar(64)	yes
`subsystem`	varchar(32)	yes
`card`	varchar(32)	yes
`value_num`	float8 null	—
`raw_json`	text null	—
`unit`	varchar(16)	—
`status`	varchar(8)	yes

Why two tables? The `parameters` table enables time-series queries like "give me all `cam_fpa_temp` values between T1 and T2" without parsing hex. The `packets` table holds the raw wire data for forensic re-decode.

Dual-write pattern

```
async def on_packet(raw, ts):
    pkt = decode(raw, ts)
    await writer.write_live(pkt)                    # Redis
    async with SessionLocal() as db:                # Postgres
        await writer.write_playback(db, pkt)
        await db.commit()
    await manager.broadcast(pkt)                    # WebSocket
```

For production, batch the Postgres write (buffer N packets, flush every 100ms). For dev, write-through is fine.

Retention and cleanup

```
# Trim Redis
docker exec tm-redis redis-cli XTRIM tm.live MAXLEN ~ 5000

# Purge old Postgres rows
docker exec tm-postgres psql -U tm -d tmdb -c \
  "DELETE FROM packets WHERE ts < NOW() - INTERVAL '7 days';"

# Vacuum
docker exec tm-postgres psql -U tm -d tmdb -c "VACUUM ANALYZE packets,
parameters;"
```

---

9. The Simulator

Until the real WFI-AI-CFI arrives, the backend can run an internal simulator that generates realistic TM.

Files

File	Purpose
`app/sim/spacecraft_model.py`	`SimState` dataclass — holds every field as a Python attribute; `tick()` advances one time step
`app/sim/packet_builder.py`	Turns a `SimState` into 18 CCSDS frames, one per APID
`app/sim/tcp_emitter.py`	Connects to the backend's own TCP listener and pushes frames at 10 Hz

How it starts

In `main.py`'s `lifespan`, if `settings.enable_sim` is True:

```
from .sim.tcp_emitter import start_simulator
asyncio.create_task(
    start_simulator(settings.tm_listen_host, settings.tm_listen_port,
rate_hz=settings.sim_rate_hz)
)
```

The simulator opens a TCP connection to `127.0.0.1:9100` (the backend's own listener). Frames round-trip through the exact same ingestion → decode → store → broadcast path used by real TM.

How to tune it

Change the rate

`.env`:

```
SIM_RATE_HZ=10      # ← 10 Hz = 10 batches/s = 30 packets/s (3 packets per
tick × 10 Hz...
                    #    actually 18 packets × 10 Hz = 180 pkt/s)
```

> **Packet rate math:** `packet_builder.build_all()` emits **18 frames per tick** (one per APID). At the default `SIM_RATE_HZ=10`, the simulator produces **180 packets per second** (18 × 10). The Live page counter reflects this aggregate rate, not the per-tick count.
>
> To reproduce a specific aggregate rate, divide by 18:

Aggregate pkt/s	SIM_RATE_HZ
18	1
90	5
180	10 (default)
360	20
900	50

Change the drift model

Edit `spacecraft_model.py`'s `tick()`. Example — make the FPA temp swing more aggressively:

```
# Before
self.cam_fpa_temp = -20 + 2 * math.sin(0.02 * t) + random.uniform(-0.2,
0.2)

# After — 5× swing amplitude, 10× faster
```

```
self.cam_fpa_temp = -20 + 10 * math.sin(0.2 * t) + random.uniform(-0.5,
0.5)
```

Force a fault to trigger an alarm

Add to `SimState`:

```
self.inject_gyro_high = True   # drifts ai_soc_temp toward HTL
```

Or manipulate a value directly:

```
def tick(self, dt=0.1):
    ...
    if self.inject_fpa_high:
        self.cam_fpa_temp += 0.5   # walk into HTL
```

Add a new field

1. Add the field to `parameters.yaml` with a name, offset, type
2. Add the matching attribute to `SimState.__init__`
3. Update `tick()` to mutate it
4. The packet builder automatically packs it

No code changes needed in `packet_builder.py` — it iterates `spec["fields"]` from the registry.

Change the initial state

Edit `SimState.__init__` — all fields have defaults. E.g.:

```
cam_fpa_temp: float = -20.0     # initial value
```

Testing the simulator

```
# Look at the log — you should see:
#   INFO app.sim.tcp_emitter — Simulator connected to 0.0.0.0:9100

# Verify packets are flowing
docker exec -it tm-redis redis-cli XLEN tm.live
docker exec -it tm-postgres psql -U tm -d tmdb -c "SELECT count(*) FROM
packets;"
```

Disable the simulator

`.env`:

```
ENABLE_SIM=false
```

External feed (no simulator)

If you have a real TM source, disable the simulator and point the source at:

- **TCP:** the source connects to `localhost:9100`
- **UDP:** the source sends datagrams to `localhost:9200`

Or send a test batch manually:

```
python -m scripts.send_test_udp --host 127.0.0.1 --port 9200 --count 200 --
rate 10
```

---

10. UI Reference — Pages and Components

Route table

Page ID	Route (in `uiStore`)	Component	Purpose
`dashboard`	Sidebar "Live"	`LivePage`	Realtime monitoring
`trends`	"Trends"	`TrendsPage`	Multi-param time series
`storage`	"Storage"	`StoragePage`	DB query + CSV export
`alarms`	"Alarms"	`AlarmsPage`	Full alarm log
`inspector`	"Inspector"	`InspectorPage`	Hex + decoded fields
`playback`	—	`PlaybackPage`	Time-window replay
`links`	—	`LinksPage`	RS-422 detail view

Live dashboard (screenshot 3 from earlier)

```
┌── Status bar ──────────────────────────────────────────────────┐
│ 🛰 WFI-AI-CFI · LIVE · T+00:08:02 · 331,131 pkt · [🔍][⚙][👤] │
├── Sidebar ──┬── Main content ──────────────────────────────────┤
│ Live        │ ┌── RS-422 Link Health strip ──────────────────┐ │
│ Trends      │ │ 7 link cards, each with state/CRC/latency    │ │
│ Storage     │ └──────────────────────────────────────────────┘ │
│ Alarms   (2)│ ┌── 4 KPI tiles ───────────────────────────────┐ │
│ Inspector   │ │ Camera FPA · AI SoC · 28V bus · Unit mode    │ │
│             │ └──────────────────────────────────────────────┘ │
│ SUBSYSTEMS  │ ┌── Thermal Trends chart ─┬── Alarm rail ──────┐ │
```

```
  | • Camera    | | Recharts multi-line    | 2 active WARN       ||
  | • CDPM (P)  | | rolling window         |                     ||
  | • CDPM (R)  | └────────────────────────┴─────────────────────┘|
  | • CTPU (P)  | ┌── Packet Decoder ──────────────────────────────┐|
  | • CTPU (R)  | | [APID dropdown] HEX DUMP | DECODED FIELDS     ||
  | • Unit      | └────────────────────────────────────────────────┘|
  └─────────────┴──────────────────────────────────────────────────┘
```

Component responsibilities:

Component	Reads from	Purpose
`StatusBar`	`useTelemetry.link`, `.totalReceived`	Connection state + counters
`Sidebar`	`useTelemetry.meta`, `.alarms`, `useFilters`, `useUiStore`	Navigation + filters
`LinkHealthStrip`	`useTelemetry.meta.links`, `.latestByApid`	RS-422 ribbon
`KpiCards`	`useTelemetry.meta`, `.latestByParam`	4 selected params
`LiveChart`	`useTelemetry.packets`	Thermal trends (throttled 2 Hz)
`AlarmRail`	`useTelemetry.alarms`	Active warnings/crits
`PacketDecoder`	`useTelemetry.packets`, `.meta`, `useFilters`	Hex + decoded view

Inspector page

Same `PacketDecoder` but with a wider layout and the `FilterBar` on top. Filtered by the sidebar subsystem selection.

Storage page

Datetime range pickers → `GET /api/history` → table of `(ts, apid, subsystem, card, crc_ok)`.

Trends page

Parameter picker (checkbox chips), `MultiSeriesChart` with up to 10 series, `TrendSparkTable` snapshot table.

---

11. Hex Inspector & Packet Inspector Usage

What the hex dump shows

Each line of the hex dump is:

```
<offset>  <16 bytes hex>  <ascii rendering>
```

Example from the screenshot:

```
0000  08 c1 00 f7 b2 00 25 c1 92 29 e1 c1 21 1f 15 40   ....H%....?!..@
0010  a0 00 00 01 42 c8 00 00 00 06 34 7a 43 a0 00 00   ...B.....4zC...
```

Breakdown of the first line:

Bytes	Field	Value
`08`	CC	0x08 (telemetry, no sec hdr)
`c1 00`	APID+flags	APID = 0x100, flags = 0xC0 (top 5 bits)
`f7 b2`	SEQ+flags	SEQ = 0x3FB2, flags = 0xC0
`00 25`	Length	0x25 = 37 → payload = 38 bytes
`c1 92 29 e1 ...`	Payload	38 bytes of packed fields
`43 a0`	CRC	0x43A0

The green **CRC OK** badge is computed by the decoder from these bytes — the inspector shows you the same bytes plus the field interpretation.

How to study packets

Step 1 — pick an APID. Use the dropdown top-right. Each option shows `0xAAA · SUBSYS/CARD`.

Step 2 — read the header rows. `CC`, `APID`, `Seq`, `Len` are always the first four decoded fields.

Step 3 — correlate with the hex. The payload starts at byte `7` in the dump. The first decoded field (e.g. `cam_fpa_temp`) is at offset 0 in the payload — so it's at byte 7 in the hex dump. Field `cam_ccd_temp` at offset 4 is at byte 11 = row 0, column 11.

Step 4 — verify manually. Pull the 4 bytes at the offset, interpret as big-endian float:

```
import struct
# bytes from hex dump at offset 7 (payload start)
struct.unpack(">f", bytes.fromhex("c19229e1"))[0]
# → -18.27 (matches cam_fpa_temp display)
```

Step 5 — check the status badge. `OK`/`WARN`/`CRIT` per field vs `LTL/LTH/HTL/HTH` from the dictionary. The limits column on the right side of each row shows them.

Step 6 — compare to live chart. The KPI card above shows the same value the inspector decodes — they should match. If they don't, something's wrong with the decoder.

Filtering by subsystem

Click a subsystem in the sidebar → `filterStore.subsystems` gains that key → `PacketDecoder` filters its APID dropdown to only that subsystem's cards. The header shows `FILTERED · 1 SUBSYSTEM`.

Uploading a captured `.bin` for offline inspection

1. Inspector page → **Upload .bin/.dump**
2. Backend `/api/upload` runs the depacketizer over the raw file
3. Returns `{count: N, packets: [...]}`
4. Decoded packets show in the response

Currently the upload endpoint returns them inline. To make the UI show them, extend `UploadDumpButton` to push `result.packets` into the telemetry store.

---

12. REST & WebSocket API

HTTP endpoints

Method	Path	Purpose
GET	`/api/meta`	TM dictionary (subsystems, links, apids)
GET	`/api/links`	Just the RS-422 link definitions
GET	`/api/packets?apid=&subsystem=&limit=`	Recent packets from DB (default 100)
GET	`/api/history?start=&end=&subsystem=&limit=`	Time-windowed packet list
POST	`/api/upload`	Multipart `.bin`/`.dump` upload → decoded
GET	`/api/alarms`	Active alarms (in-memory)
POST	`/api/alarms/{id}/ack`	Acknowledge an alarm
GET	`/docs`	Swagger UI

WebSocket endpoints

Path	Purpose
`/ws/telemetry`	Live stream: server pushes `{t, packets:[...]}` at ingest rate
`/ws/playback?start=&end=&speed=`	Replay: server emits packets at `speed` multiplier, then closes

WS payload schema — matches `frontend/src/types/packet.ts`:

```
{
  "t": 1727901234.567,
  "packets": [
    {
      "cc": 8,
      "apid": 256,
      "apid_name": "Camera Electronics Card",
      "subsystem": "CAMERA",
      "card": "CAM_ELEC",
      "hw_class": "HW",
      "seq": 14397,
      "length": 37,
      "crc_ok": true,
      "payload_hex": "...",
      "raw_hex": "...",
      "fields": {
        "cam_fpa_temp": {
          "value": -18.27,
          "unit": "C",
          "limits": [-40, 20],
          "status": "OK",
          "enum": null
        },
        "cam_clock_locked": {
          "value": 1,
          "unit": "-",
          "limits": [null, null],
          "status": "OK",
          "enum": {"0": "UNLOCKED", "1": "LOCKED"}
        }
      },
      "timestamp": 1727901234.567
    }
  ]
}
```


---

13. Mapping TM → UI → Backend

End-to-end trace for `cam_fpa_temp`:

Layer	Artifact	Reference
ICD	The value is defined as "FPA temperature, C, ±0.1 precision"	(pending ICD)
Dictionary	`parameters.yaml → CAMERA → CAM_ELEC → cam_fpa_temp`	offset 0, f32, °C, LTL −40, LTH −5, HTL 15, HTH 20
Simulator	`SimState.cam_fpa_temp` mutated in `tick()`	`−20 + 2·sin(0.02t) + noise`

Layer	Artifact	Reference
Packet builder	Iterates `spec["fields"]` → packs at offset 0 as f32	`packet_builder._build`
Wire	`0x100` APID, bytes 7–10 of the frame = `f32` big-endian	CCSDS §4
Decoder	`decoder.decode` reads offset 0, casts f32, applies limits	`_status()`
Store (Redis)	JSON blob in `tm.live` stream	`StorageWriter.write_live`
Store (Postgres)	Row in `packets` + row in `parameters(name='cam_fpa_temp', value_num=…)`	`write_playback`
WS broadcast	`manager.broadcast(pkt)`	`manager.broadcast`
Frontend store	`telemetryStore.ingest(batch)` → `latestByParam["256:cam_fpa_temp"]`	`ingest`
UI (KPI card)	`KpiCards` looks up `latestByParam` at `meta.subsystems.CAMERA.cards.CAM_ELEC.apid + ":cam_fpa_temp"`	`KpiCards`
UI (chart)	`LiveChart` filters `packets` for `apid === 0x100`, extracts `fields.cam_fpa_temp.value`	throttled 2 Hz
UI (inspector)	`PacketDecoder` shows the last 0x100 packet's hex + decoded fields	`DecodedFieldsList`
Alarm	`ingest` checks `field.status !== "OK"` → creates alarm in `alarms[]`	`AlarmRail`, `AlarmsPage`

To trace any other parameter, replace `cam_fpa_temp` with its name and start from the dictionary entry.

---

14. The ICD & Interfaces

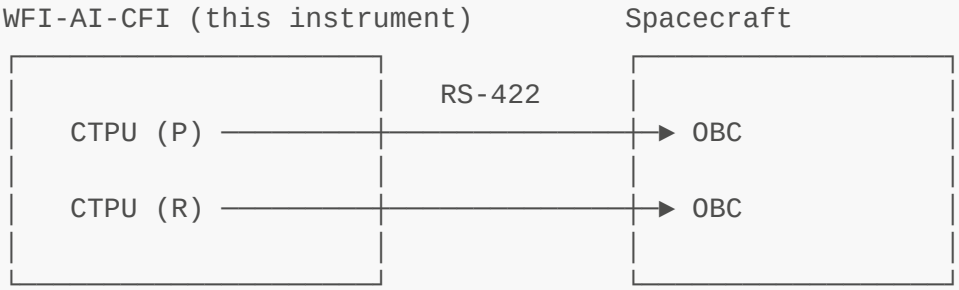
Current state (as of this writing)

The **ICD is not frozen**. The APIDs, offsets, types, and limits in `parameters.yaml` / `links.yaml` are **informed guesses** based on:

- Standard spacecraft subsystem TM patterns
- The instrument's physical architecture (three cards per CDPM, two per CTPU, etc.)
- Typical RS-422 link health metrics

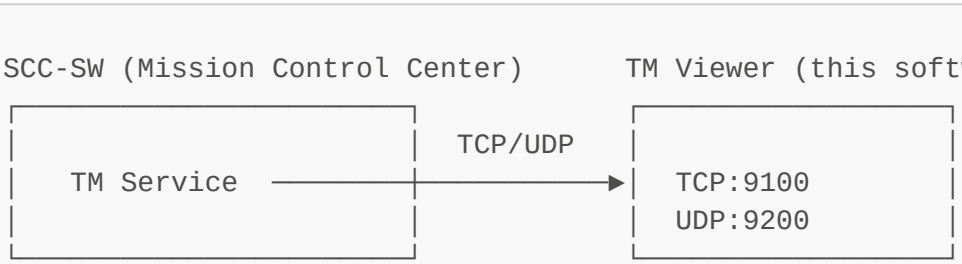
As the ICD firms up, edit the two YAML files and reload — no code changes needed.

The instrument ↔ spacecraft interface



Only CTPU talks to the spacecraft. The CDPMs, Camera, and all other cards are internal to the instrument.

The SCC-SW ↔ TM Viewer interface



The SCC-SW's TM service pushes already-depacketized (or raw) frames to our TCP/UDP listener. Our stack does the rest.

Per-subsystem wiring

Subsystem	Interface to rest of instrument	TM content
Camera	RS-422 to CDPM (P), RS-422 to CDPM (R)	FPA/CCD temps, bias, frame counter
CDPM (P)	RS-422 to Camera, RS-422 to CTPU (P), RS-422 cross-strap to CDPM (R)	FPGA + AI + THCC
CDPM (R)	Same, alternate	FPGA + AI
CTPU (P)	RS-422 to CDPM (P), RS-422 to S/C	Power + PPC
CTPU (R)	Same, alternate	Power + PPC

Every one of these RS-422 links is monitored by a dedicated APID.

---

15. Extending the System

Add a new parameter to an existing card

1. Edit `parameters.yaml`, find the card, append a field entry with a fresh `offset`
2. If it's an int type, make sure the new offset + size doesn't overlap
3. Add the attribute to `SimState` with an initial value

4. Update `SimState.tick()` to mutate it
5. Restart the backend

The decoder, storage, WS, and frontend all pick it up automatically.

Add a new card

1. Edit `parameters.yaml`, add a `<NEW_CARD>` under the appropriate subsystem with a fresh APID
2. Add its `fields` list
3. Add matching attributes to `SimState` for each field
4. Update `SimState.tick()`
5. Restart the backend

The registry validates:

- APID uniqueness
- No overlapping field offsets
- All types are valid

Add a new subsystem

1. Same as "add a card" but also add a new top-level `<NEW_SUBSYSTEM>` key
2. Choose a distinct color for the UI accent
3. Restart

The sidebar, KPI cards, inspector, trends — everything picks up the new subsystem.

Change an APID

1. Edit `parameters.yaml`, change `apid: 0xXXX`
2. Restart

The old APID's packets from the DB will still be there but won't decode until you query them with the old dictionary. **Do not change APIDs after the ICD is frozen.**

Add an OEM packet format

Currently packets use a simplified CCSDS-like frame. When you're ready to support OEM packet formats:

1. Add a `format:` field to each card in `parameters.yaml` (`ccsds` | `oem`)
2. Add `app/tm_service/inspector/oem_decoder.py` that mirrors `decoder.py`
3. Modify `decoder.decode()` to dispatch based on the card's format
4. Add OEM-specific fields to the payload layout

See §16 for details.

Point the viewer at a real TM source

1. Set `ENABLE_SIM=false` in `.env`
2. Restart the backend
3. Configure the real TM source to connect to:

- `localhost:9100` (TCP), **or**
- `localhost:9200` (UDP)

4. If the source sends OEM packets instead of CCSDS, implement the OEM decoder first

Add a new REST endpoint

1. Create `app/api/routes_<name>.py` with an `APIRouter`
2. Include it in `main.py`: `app.include_router(<name>_router)`
3. Document it in this file under §12

Add a new frontend page

1. Create `src/pages/<Name>Page.tsx`
2. Add the ID to `Page` type in `src/store/uiStore.ts`
3. Add a `NavItem` entry in `Sidebar.tsx`
4. Add the conditional render in `App.tsx`

Change the KPI cards on the Live page

Edit `FEATURED` in `src/components/live/KpiCards.tsx`:

```
const FEATURED = [
  { sub: "CAMERA", card: "CAM_ELEC",   param: "cam_fpa_temp" },
  { sub: "CDPM_P", card: "CDPM_P_AI",  param: "ai_soc_temp" },
  { sub: "CTPU_P", card: "CTPU_P_PWR", param: "pwr_out_28v_bus_v" },
  { sub: "UNIT",   card: "UNIT_HK",    param: "unit_mode" },
];
```

Replace any entry with a different `(subsystem, card, param)` tuple.

Change the Live chart series

Edit `SERIES` in `src/components/live/LiveChart.tsx`:

```
const SERIES = [
  { apid: 0x100, field: "cam_fpa_temp",  key: "cam_fpa",  label: "...",
color: "#00E5FF" },
  { apid: 0x111, field: "ai_soc_temp",   key: "ai_soc",   label: "...",
color: "#22C55E" },
  { apid: 0x131, field: "ppc_dcdc_temp", key: "ppc_dcdc", label: "...",
color: "#FFB020" },
];
```

Add or remove entries. Up to ~10 series render cleanly.

---

16. Placeholder — OEM Packet Format (Future)

> **Status:** Planned. This section documents the intended design so implementation can proceed cleanly when the ICD specifies an OEM-based packet format.

What OEM means here

"OEM packet format" (Original Equipment Manufacturer) generally refers to a vendor-defined packet framing that carries metadata alongside the payload — commonly:

- A magic number / sync word for framing
- Version and type fields
- Header with timestamp, sequence, source ID
- Payload
- Optional checksum (varies: CRC-16, CRC-32, XOR)

Unlike CCSDS, there is no universal standard. The exact layout must come from the ICD.

Anticipated changes

Area	Change
Dictionary	New field per card: `format: ccsds | oem`
Depacketizer	New `OemDepacketizer` that recognizes sync word + length
Decoder	Dispatcher at `decoder.decode()` that routes by format
CRC	Pluggable via a `checksum: crc16-ccitt | crc32 | xor8 | none` field
Registry	Validate format-specific required fields (sync word, length offset)
Storage	`packets` table gains a `format` column
UI	Hex viewer highlights sync word / header / payload ranges with different colors

Proposed `oem.yaml` (example)

```yaml
oem:
  sync_word: 0xAA55
  header_len: 12
  fields:
    - { name: sync,     offset: 0,  type: u16, unit: "-" }
    - { name: version,  offset: 2,  type: u8,  unit: "-" }
    - { name: msg_type, offset: 3,  type: u8,  unit: "-" }
    - { name: source,   offset: 4,  type: u16, unit: "-" }
    - { name: seq,      offset: 6,  type: u32, unit: "-" }
    - { name: length,   offset: 10, type: u16, unit: "bytes" }
  checksum: crc16-ccitt
```

Migration path

1. Ship OEM support **alongside** CCSDS — do not replace

2. Each card in `parameters.yaml` declares its own format
3. Real TM from the ICD will indicate format in the payload itself; use that to route
4. Keep `decoder.py` for CCSDS, add `oem_decoder.py` for OEM, dispatch at the top

When ready to implement

Ping with the OEM ICD and I'll produce:

- `app/tm_service/inspector/oem_decoder.py`
- `app/tm_service/ingestion/oem_depacketizer.py`
- Registry schema changes
- Storage schema migration
- UI hex-viewer highlighting

---

17. Troubleshooting Map

Symptom	Likely cause	Look at	Fix
No packets in UI	Simulator not running	Backend log: `Simulator connected`	Check `ENABLE_SIM=true`; check port 9100
Packets but chart blank	Chart filter APID doesn't exist	`LiveChart.tsx SERIES`	Match APID to a real card
CRC `FAIL` in Storage	`/api/history` doesn't return `crc_ok`	`routes_history.py`	Include `crc_ok` in the response dict
White dropdown	Native `<select>` unstyled	`index.css`	Add `color-scheme: dark` + `.select-dark`
Sidebar click does nothing	Filter isn't consumed	`PacketDecoder.tsx`	Filter `meta.apids` by `filters.subsystems`
Hook error on render	Hook inside `.map`	`Sidebar.tsx`	Hoist hooks to component top
Max update depth	Store churns every tick	`LiveChart.tsx`	Throttle chart rebuild to 2 Hz
DB auth fails	Stale Postgres volume	`docker inspect tm-postgres`	Wipe volume, recreate container
Backend won't start	Port 9100/9200 in use	`lsof -i :9100`	Kill the holder
No `/api/meta` response	Backend not running	`curl localhost:8000/api/meta`	Restart uvicorn
OEM packets not decoding	Not implemented yet	—	See §16

Diagnostic command quick list

```
# Is the backend up?
curl -s http://localhost:8000/api/meta | python -m json.tool | head

# Are packets arriving?
docker exec -it tm-redis redis-cli XLEN tm.live
docker exec -it tm-postgres psql -U tm -d tmdb -c "SELECT count(*) FROM
packets;"

# What's the last packet look like?
docker exec -it tm-postgres psql -U tm -d tmdb -c \
  "SELECT ts, apid, subsystem, card, crc_ok FROM packets ORDER BY ts DESC
LIMIT 5;"

# Does the WS push?
websocat ws://localhost:8000/ws/telemetry | head -1

# Registry sanity
cd backend && python -c "from app.tm_service.registry import get_registry;
\
  r = get_registry(); print(len(r.apid_map), 'apids')"
```

Appendix A — Field Type Cheatsheet

YAML type	Python	struct fmt	Byte order	Range
u8	int	B	—	0–255
u16	int	H	big	0–65535
u32	int	I	big	0–4294967295
i8	int	b	—	−128–127
i16	int	h	big	−32768–32767
i32	int	i	big	−2147483648–2147483647
f32	float	f	big	IEEE-754 single
f64	float	d	big	IEEE-754 double

Appendix B — Status Thresholds

Threshold	Meaning	Example
LTL	Low Threshold, **Critical**	−40 °C (FPA minimum survivable)
LTH	Low Threshold, **Warning**	−5 °C (below normal operating)
HTL	High Threshold, **Warning**	15 °C (above normal operating)

Threshold	Meaning	Example
HTH	High Threshold, **Critical**	20 °C (near-damage)

Fields with only LTL/HTL: two-level alarm. Fields with all four: three-level (OK / WARN / CRIT).

Appendix C — File Inventory

Backend — TM Service

Path	Purpose	Edit when
`backend/app/tm_dictionary/parameters.yaml`	Card/field definitions	ICD updates, new params
`backend/app/tm_dictionary/links.yaml`	RS-422 link definitions	New links, changed counters
`backend/app/tm_service/registry.py`	YAML loader + validator	Schema changes
`backend/app/tm_service/ingestion/*`	Sources (TCP, UDP, file, DB)	New source type
`backend/app/tm_service/ingestion/depacketizer.py`	Stream → frame slicer	Length-field semantics change
`backend/app/tm_service/ingestion/format_map.py`	Decoder mode → `header_len` map	New decoder format added
`backend/app/main.py`	Startup wiring, decoder init	New subsystems, new sources

Backend — Inspector / Decoders

Path	Purpose	Edit when
`backend/app/tm_service/inspector/base.py`	`PacketDecoder` ABC — decoder interface	Adding methods to the interface
`backend/app/tm_service/inspector/custom_decoder.py`	Legacy 7-byte-header decoder	Custom ICD layout changes
`backend/app/tm_service/inspector/pus_decoder.py`	ECSS PUS-C decoder (via `spacepackets`)	PUS version changes, timestamp

Path	Purpose	Edit when
		length changes
`backend/app/tm_service/inspector/factory.py`	Decoder registry + `get_decoder()`	Registering a new decoder
`backend/app/tm_service/inspector/ccsds.py`	CCSDS primary-header parser	Rarely
`backend/app/tm_service/inspector/crc.py`	CRC-16-CCITT	Never

Backend — Simulator

Path	Purpose	Edit when
`backend/app/sim/spacecraft_model.py`	Simulator state + `tick()`	Testing drift models, injecting faults
`backend/app/sim/packet_builder.py`	State → frames	Rarely (only if struct packing changes)
`backend/app/sim/tcp_emitter.py`	Feeds TCP source locally	Rarely

Frontend

Path	Purpose	Edit when
`frontend/src/components/live/KpiCards.tsx`	KPI tiles	Which params show on Live
`frontend/src/components/live/LiveChart.tsx`	Live chart series	Which params trend
`frontend/src/components/live/PacketDecoder.tsx`	Hex + decoded inspector	Adding fields to the header row
`frontend/src/pages/*.tsx`	Route-level UI	Adding features per page
`frontend/src/store/telemetryStore.ts`	Runtime state + alarm engine	If ingest/alarm logic changes
`frontend/src/store/settingsStore.ts`	Theme + API config	Adding new settings
`frontend/src/hooks/useTelemetryStream.ts`	WS client with reconnect	Stream protocol changes
`frontend/src/lib/api.ts`	API base URLs, wire DTOs	Backend schema changes

Configuration

Path	Purpose	Edit when
`backend/.env`	Runtime config (ports, decoder mode, DB URLs)	Environment changes
`backend/.env.example`	Config template	Adding new config keys
`backend/requirements.txt`	Python deps	Adding libraries (e.g. `spacepackets`)
`frontend/package.json`	Node deps	Adding libraries
`frontend/tailwind.config.js`	Theme tokens, colors	Rebranding, palette changes
`frontend/src/index.css`	CSS variables (dark + light themes)	Theme adjustments

End of REFERENCE.md

> Keep this file updated as the ICD evolves. Every change to `parameters.yaml` or `links.yaml` should be reflected here so the doc remains the single source of truth for the team. When the OEM ICD arrives, replace §16 with the concrete format and update §4 accordingly.